DAY 7

# Insertion & Deletion in Arrays

DSA From Scratch to Advanced — Day-by-Day Learning Series

### Core idea

Arrays are contiguous, so changing the middle can require shifting elements.

### Pattern

Insert → shift right
Delete → shift left

### Goal

Understand the cost behind array modifications.

# Learning Roadmap

Concept → Example → Implementation → Complexity → Pattern → Problem → Solution → Takeaway

1. What insertion and deletion mean
2. Why shifting happens in arrays
3. Insert at beginning, middle and end
4. Delete from beginning, middle and end
5. C++ implementation with fixed arrays
6. std::vector and its practical behavior
7. Complexity, patterns and interview tips
8. LeetCode practice

# What Is Array Insertion?

Adding a new element at a chosen position

### Definition

Insertion means placing a new value at an index while keeping the remaining elements in order.

### Key constraint

If the position is not the end, existing elements must move one step to the right.

Example: [10, 20, 30, 40]

Insert 25 at index 2 → [10, 20, 25, 30, 40]

# Why Do We Need Insertion?

Real programs constantly add data

- Maintain an ordered collection
- Insert a value at a specific rank/position
- Build arrays dynamically through vectors
- Support algorithms that maintain a sorted sequence
- Understand why some data structures are better than arrays for frequent middle insertion

**Trade-off**

Fast random access, slower middle insertion.

# Insertion: The Core Pattern

Shift from right → left, then place the new value

```
for (int i = n; i > pos; --i)
    arr[i] = arr[i - 1];

arr[pos] = value;
++n;
```

**Step 1**

Start at the last element

**Step 2**

Shift right

**Step 3**

Write at pos

# Insertion Example

Insert 25 at index 2

```
Before:  [10, 20, 30, 40, _]
Index:    0   1   2   3   4

Shift:   30 → 40   (move right)
         20 stays

After:   [10, 20, 25, 30, 40]
                ↑
             new value
```

We move elements from the back so we do not overwrite data.

Number of shifted elements = n - pos.

# Insertion at Different Positions

The position determines the cost

### Beginning — O(n)

Insert at index 0.
Every existing element shifts right.

### Middle — O(n)

Roughly half the array may shift.

### End — O(1)*

If capacity/space is available, append needs no shifting.

*For a fixed array, you still need free capacity. For std::vector, push_back is amortized O(1).

# What Is Array Deletion?

Removing an element and preserving order

### Definition

Deletion removes the element at a chosen index.

### Key constraint

If the deleted element is not last, later elements shift one position to the left.

Example: [10, 20, 30, 40]

Delete index 1 → [10, 30, 40]

# Deletion: The Core Pattern

Shift from left → right, then reduce the logical size

```
for (int i = pos; i < n - 1; ++i)
    arr[i] = arr[i + 1];

--n;
```

**Step 1**

Start at pos

**Step 2**

Shift left

**Step 3**

Decrease n

# Deletion Example

Delete 30 from index 2

```
Before:  [10, 20, 30, 40, 50]
Index:    0   1   2   3   4

Shift left: 40 → 30's position
            50 → next position

After:   [10, 20, 40, 50]
                  ↑
               gap removed
```

Number of shifted elements = n - pos - 1.

# Deletion at Different Positions

Again, position controls the work

### Beginning — O(n)

Deleting index 0 shifts every remaining element left.

### Middle — O(n)

Later elements must shift left to close the gap.

### End — O(1)

Deleting the last logical element only decreases size.

# Complete C++ Implementation

Fixed-size array: insertion + deletion

```
#include <iostream>
using namespace std;

void insertAt(int arr[], int &n, int pos, int value) {
    for (int i = n; i > pos; --i)
        arr[i] = arr[i - 1];
    arr[pos] = value;
    ++n;
}

void deleteAt(int arr[], int &n, int pos) {
    for (int i = pos; i < n - 1; ++i)
        arr[i] = arr[i + 1];
    --n;
}
```

# Safety Checks in Real Code

Never assume the index is valid

```
bool validInsert(int pos, int n, int capacity) {
    return pos >= 0 && pos <= n && n < capacity;
}

bool validDelete(int pos, int n) {
    return pos >= 0 && pos < n;
}
```

Insertion allows pos == n (append).

Deletion requires pos < n.

Always ensure capacity before shifting during insertion.

# std::vector: The Practical C++ Choice

Dynamic arrays hide capacity management

```
vector<int> a = {10, 20, 30, 40};

a.insert(a.begin() + 2, 25);
// [10, 20, 25, 30, 40]

a.erase(a.begin() + 3);
// [10, 20, 25, 40]

a.push_back(50);
```

vector automatically manages memory and size.

insert/erase in the middle still require shifting → O(n).

# Fixed Array vs std::vector

Know what the abstraction is doing

## Fixed Array

- Fixed capacity
- Manual size tracking
- Manual shifting
- Excellent for learning memory layout
- No automatic resizing

## std::vector

- Dynamic capacity
- size() tracks logical elements
- insert()/erase() available
- push_back() is amortized O(1)
- Reallocation can cost O(n)

# Time & Space Complexity

The most important interview takeaway

### Insertion

Beginning: O(n)
Middle: O(n)
End: O(1)*

### Deletion

Beginning: O(n)
Middle: O(n)
End: O(1)

### Extra Space

O(1) auxiliary space
when shifting in-place.

*For vector, push_back is amortized O(1), but reallocation can occasionally take O(n).

# Why Is Insertion O(n)?

Think in terms of elements moved

If inserting at index 0 in n elements → n elements may shift.

If inserting at index pos → n - pos elements shift.

So the worst case is O(n).

Arrays are contiguous: preserving order is what creates the shifting cost.

**Mental model**

Random access ≠ cheap modification

# Why Is Deletion O(n)?

Removing one item can create a gap

Delete at index pos.

All elements after pos must move left.

Number moved = n - pos - 1.

Worst case: delete the first element → O(n).

Delete the last element → no shifting → O(1).

# Pattern: Shift to Preserve Order

A reusable DSA pattern

### Insertion Pattern

RIGHT → LEFT
Shift elements one position right.
Then write the new value.

### Deletion Pattern

LEFT → RIGHT
Shift later elements left.
Then reduce logical size.

Whenever an array operation changes the middle, ask: “Which elements must move to preserve order?”

# LeetCode Practice #1

27 — Remove Element

Goal: remove all occurrences of a given value in-place.

Key idea: use a write pointer to overwrite unwanted values.

Time: O(n) • Extra space: O(1)

Pattern connection: deletion without repeatedly shifting every element.

```
for (int x : nums) {
    if (x != val) nums[k++] = x;
}
return k;
```

# LeetCode Practice #2

26 — Remove Duplicates from Sorted Array

Goal: remove duplicates in-place from a sorted array.

Use two pointers: one scans, one writes the next unique value.

Time: O(n) • Extra space: O(1)

Pattern connection: compact the array by overwriting positions.

```
int k = 1;
for (int i = 1; i < nums.size(); ++i)
    if (nums[i] != nums[k - 1])
        nums[k++] = nums[i];
return k;
```

# LeetCode Practice #3

88 — Merge Sorted Array

Goal: merge two sorted arrays into nums1 in-place.

Use three pointers from the end.

Why from the end? It avoids overwriting useful values in nums1.

Time: O(m + n) • Extra space: O(1)

Pattern connection: controlled in-place movement.

```
int i=m-1, j=n-1, k=m+n-1;
while (j >= 0) {
    if (i >= 0 && nums1[i] > nums2[j]) nums1[k--] = nums1[i--];
    else nums1[k--] = nums2[j--];
}
```

# Interview Tip

Don't just say “O(n)” — explain why

### Weak answer

“Insertion in an array is O(n).”

### Strong answer

“Insertion can be O(n) because elements after the insertion point must shift right to preserve order.”

Bonus: mention that appending to vector is amortized O(1), while middle insertion remains O(n).

# Day 7 Takeaway

What you should remember

- ✅ Insert → shift right → place value → increase size
- ✅ Delete → shift left → decrease size
- ✅ Middle modifications are O(n) because of shifting
- ✅ In-place shifting uses O(1) extra space
- ✅ vector makes resizing easier, not middle insertion faster
- ✅ Two-pointer compaction is a powerful in-place pattern

> **One-line memory hook**
>
> Array = fast access, expensive middle edits.

# Tomorrow: Searching in Arrays

Day 8 — Linear Search & Binary Search

### You learned today

How arrays insert/delete and why shifting determines complexity.

### Tomorrow

Linear Search → Binary Search → when to use each → complexity → problems.

Keep practicing: implement insertAt() and deleteAt() without looking at the code.

Follow the series for the complete journey from arrays to advanced DSA.